

React.js Interview Guide

Top 50 Questions · Senior Frontend Interviewer Edition

50 Total Qs

■ High Priority

★ Top 10 Must Know

■ Tricky Edge Cases

★ TOP 10 MUST-KNOW QUESTIONS

01	What is the Virtual DOM and how does React use it?
02	What are React Hooks? Name the most common ones.
03	What is the difference between useEffect and useLayoutEffect?
04	What is the difference between useMemo and useCallback?
05	How does React.memo work and when should you use it?
06	What is prop drilling and how do you avoid it?
07	Explain the React component lifecycle.
08	What is the difference between controlled and uncontrolled components?
09	How does the useReducer hook work? When would you prefer it over useState?
10	What are React keys and why are they important in lists?

■ BASICS

01 **HIGH** [Core] What is React and why is it used?

ANSWER

React is a JavaScript library for building UI components. It uses a Virtual DOM to efficiently update the real DOM, making apps fast and scalable. It's component-based, so UI is reusable and maintainable.

■ FOLLOW-UP:

Why choose React over vanilla JS or Angular?

02 **HIGH** [Core] What is JSX?

ANSWER

JSX is a syntax extension that looks like HTML inside JavaScript. Babel compiles it to `React.createElement()` calls. It makes writing UI more intuitive and readable.

■ FOLLOW-UP:

Can you write React without JSX? Show an example.

03 **HIGH** ★ TOP 10 [Core] What is the Virtual DOM and how does React use it?

ANSWER

Virtual DOM is a lightweight JS object copy of the real DOM. When state changes, React creates a new Virtual DOM, diffs it with the old one (reconciliation), and only updates what changed in the real DOM — making updates fast.

■ FOLLOW-UP:

What is reconciliation? What algorithm does React use?

04

HIGH

[Core] What is the difference between state and props?

ANSWER

Props are read-only data passed from parent to child. State is local, mutable data managed inside a component. When state or props change, the component re-renders.

■ FOLLOW-UP:

Can a child component modify props? What happens if you try?

05

HIGH

★ TOP 10 [Hooks] What are React Hooks? Name the most common ones.

ANSWER

Hooks are functions that let you use state and lifecycle features in functional components. Most common: `useState`, `useEffect`, `useRef`, `useContext`, `useMemo`, `useCallback`, `useReducer`.

■ FOLLOW-UP:

Why were Hooks introduced? What problem did they solve over class components?

06

MED

[Core] What is the difference between functional and class components?

ANSWER

Functional components are plain JS functions that return JSX. Class components extend `React.Component` and use lifecycle methods. Hooks replaced the need for class components in most cases.

■ FOLLOW-UP:

Are class components still relevant? Do companies still use them?

07

HIGH

★ TOP 10 [Forms] What are controlled vs uncontrolled components?

ANSWER

Controlled: form data is handled by React state (value + onChange). Uncontrolled: form data is handled by the DOM itself via refs. Controlled components give more control over validation and submission.

■ FOLLOW-UP:

When would you prefer uncontrolled components?

08

HIGH

★ TOP 10 [Core] What is the purpose of keys in lists?

ANSWER

Keys help React identify which list items changed, were added, or removed during re-renders. They must be unique among siblings. Using index as key can cause bugs with reordering.

■ FOLLOW-UP:

Why is using array index as key a bad practice? Show a real bug scenario.

09

MED

[Core] What is one-way data flow in React?

ANSWER

Data in React flows from parent to child through props only. Child components cannot directly modify parent state. This makes the app predictable and easier to debug.

■ FOLLOW-UP:

How do you pass data from child to parent then?

10

LOW

[Core] What is the difference between React and ReactDOM?

ANSWER

React is the core library for building components. ReactDOM is responsible for rendering those components to the browser's DOM. They're separate to support React Native (no DOM needed).

■ FOLLOW-UP:

What does ReactDOM.createRoot do differently from ReactDOM.render?

■ INTERMEDIATE

11

HIGH

★ TOP 10 [Hooks] What is the difference between useEffect and useLayoutEffect?

ANSWER

useEffect runs asynchronously after the DOM has painted — good for data fetching. useLayoutEffect runs synchronously before the browser paints — use it for DOM measurements or animations to prevent flicker.

■ FOLLOW-UP:

Can overusing useLayoutEffect affect performance? Why?

12

HIGH

★ TOP 10 [State] What is prop drilling and how do you avoid it?

ANSWER

Prop drilling is when you pass props through many intermediate components that don't use them, just to get data to a deeply nested child. Avoid it using Context API, Zustand, Redux, or component composition.

■ FOLLOW-UP:

What are the downsides of Context API for state management?

13

HIGH

★ TOP 10 [Lifecycle] Explain React component lifecycle (modern hooks way).

ANSWER

Mount: `useEffect(()=>{}, [])` runs once after first render. Update: `useEffect(()=>{}, [dep])` runs when `dep` changes. Unmount: return a cleanup function inside `useEffect`. This maps to `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`.

■ FOLLOW-UP:

What happens if you don't return a cleanup in `useEffect` for an event listener?

14

HIGH

★ TOP 10 [Hooks] How does `useReducer` work? When would you prefer it over `useState`?

ANSWER

`useReducer(reducer, initialState)` manages state with a reducer function `(state, action) => newState`. Prefer it when state has complex logic, multiple sub-values, or the next state depends on the previous one. Think of it as a mini Redux.

■ FOLLOW-UP:

Can you combine `useReducer` with `Context` to replace Redux?

15

HIGH

[State] What is `Context` API and when should you use it?

ANSWER

`Context` provides a way to pass data globally without prop drilling. Use it for: auth state, theme, language. Avoid using it for frequently changing state — every consumer re-renders when context value changes.

■ FOLLOW-UP:

How would you optimize `Context` to prevent unnecessary re-renders?

16

HIGH

★ TOP 10 [Performance] What is React.memo and when should you use it?

ANSWER

React.memo is a HOC that memoizes a functional component — it only re-renders if its props change. Use it on pure components that receive the same props often, like list items or UI widgets.

■ FOLLOW-UP:

Does React.memo do deep comparison or shallow? How do you customize it?

17

HIGH

★ TOP 10 [Performance] What is the difference between useMemo and useCallback?

ANSWER

useMemo memoizes a computed value — returns the result. useCallback memoizes a function definition — returns the function. Use useMemo for expensive calculations, useCallback to prevent child re-renders caused by new function references.

■ FOLLOW-UP:

Is it always beneficial to use useMemo/useCallback? What's the cost?

18

HIGH

[Hooks] What are custom hooks and how do you create one?

ANSWER

Custom hooks are functions starting with 'use' that extract reusable logic using built-in hooks. Example: useFetch, useLocalStorage, useDebounce. They help keep components clean and logic reusable.

■ FOLLOW-UP:

Can two components sharing the same custom hook share state?

19

MED

[Performance] What is lazy loading in React?

ANSWER

React.lazy() dynamically imports a component only when it's needed. Combined with Suspense, it shows a fallback while loading. This reduces the initial bundle size — critical for performance.

■ FOLLOW-UP:

What are the limitations of React.lazy? Can you lazy load named exports?

20

MED

[Hooks] What is the useRef hook used for?

ANSWER

useRef returns a mutable object { current: value } that persists across renders without causing re-renders. Use it for: accessing DOM elements directly, storing previous values, or persisting timers/intervals.

■ FOLLOW-UP:

What's the difference between useRef and createRef?

21

MED

■ TRICKY [State] Why does React state immutability matter for nested objects?

ANSWER

Shallow copy copies only top-level properties; nested objects are still referenced. React's state updates should be immutable — if you mutate a nested object directly without spreading, React won't detect the change and won't re-render.

■ FOLLOW-UP:

How would you update a nested object in state immutably?

22

LOW

[Routing] What is the difference between BrowserRouter and HashRouter?

ANSWER

BrowserRouter uses the HTML5 history API for clean URLs like /about. HashRouter uses URL hash (#/about) — no server config needed, useful for static hosting. BrowserRouter needs server-side support for direct URL access.

■ FOLLOW-UP:

How do you handle 404s with BrowserRouter on a static server?

■ ADVANCED

23

MED

[React 18] What is Concurrent Mode / Concurrent React?

ANSWER

Concurrent Mode (React 18) lets React work on multiple tasks simultaneously. It can pause, resume, or abandon renders. Features like `useTransition` and `Suspense` leverage this to keep the UI responsive during heavy updates.

■ FOLLOW-UP:

What is the difference between `startTransition` and `setTimeout` for deferring updates?

24

MED

[React 18] What are `useTransition` and `useDeferredValue`?

ANSWER

`useTransition` marks a state update as non-urgent (like a search filter) — React can interrupt it to handle urgent updates (like typing). `useDeferredValue` defers a value update similarly, useful when you don't own the state update code.

■ FOLLOW-UP:

How would you use `useTransition` to optimize a large search/filter list?

25

MED

[Patterns] What are Higher-Order Components (HOC)?

ANSWER

A HOC is a function that takes a component and returns a new enhanced component. Example: `withAuth(Component)`. They were popular pre-hooks for cross-cutting concerns like logging, auth, theming. Now often replaced by custom hooks.

■ FOLLOW-UP:

What problems can HOCs cause? (hint: wrapper hell)

26

LOW

[Patterns] What are render props?

ANSWER

A render prop is a prop whose value is a function that returns JSX. The component calls that function to render UI. It's a pattern for sharing logic between components without HOCs.

■ FOLLOW-UP:

Compare render props vs custom hooks for sharing logic.

27

MED

[Performance] What is code splitting and how do you implement it in React?

ANSWER

Code splitting breaks your app bundle into smaller chunks loaded on demand. In React, use `React.lazy` + `Suspense`, or `dynamic import()`. Tools like Webpack and Vite handle the splitting automatically.

■ FOLLOW-UP:

What is route-based vs component-based code splitting?

28

MED

[Internals] What is the Fiber architecture in React?

ANSWER

React Fiber (React 16+) is a complete rewrite of the reconciliation engine. It breaks rendering work into small units (fibers) that can be paused and resumed. This enables Concurrent Mode, Suspense, and incremental rendering.

■ FOLLOW-UP:

How does Fiber enable priority-based rendering?

29

HIGH

■ TRICKY [Hooks] What is the problem with stale closures in useEffect?

ANSWER

If `useEffect` captures a value from the outer scope (like `state`) but doesn't list it in `deps`, it reads an old stale value. This is a stale closure bug. Fix it by adding the value to the dependency array, or using `useRef` for mutable values.

■ FOLLOW-UP:

Demonstrate a stale closure bug with a counter example.

30

MED

■ TRICKY [Core] How does `React.StrictMode` affect your app?

ANSWER

`StrictMode` double-invokes certain functions (like `render`, `state updaters`) in development to detect side effects. It highlights deprecated APIs and warns about unsafe patterns. It has zero impact on production builds.

■ FOLLOW-UP:

Why might your `useEffect` run twice in development? Is this a bug?

31

MED

[SSR] What is SSR and how does Next.js enable it?

ANSWER

SSR renders React on the server and sends HTML to the client — improving initial load time and SEO. Next.js enables it via `getServerSideProps` or React Server Components. The client then 'hydrates' the HTML with React.

■ FOLLOW-UP:

What is the difference between SSR, SSG, and ISR in Next.js?

32

MED

■ TRICKY [SSR] What is hydration in React?

ANSWER

Hydration is when React attaches event listeners to server-rendered HTML, making it interactive. It's faster than re-rendering from scratch. Mismatches between server and client HTML cause 'hydration errors'.

■ FOLLOW-UP:

What causes a hydration mismatch and how do you fix it?

33

HIGH

■ TRICKY [React 18] Explain React's automatic batching in React 18.

ANSWER

In React 18, all state updates are automatically batched everywhere — inside `setTimeout`, promises, and native event handlers. Earlier, batching only happened in React event handlers. This reduces unnecessary re-renders significantly.

■ FOLLOW-UP:

How do you opt out of automatic batching if needed?

34

LOW

[Core] What is the difference between imperative and declarative programming in React?

ANSWER

Imperative: you describe HOW to do something (manipulate DOM steps). Declarative: you describe WHAT you want (React handles the DOM updates). React is declarative — you just express the desired state and React figures out the DOM updates.

■ FOLLOW-UP:

What are the advantages of declarative UI in large teams?

■ CODING / PRACTICAL

35

HIGH

[Custom Hook] Write a custom useFetch hook that handles loading, error, and data states.

CODE

```
function useFetch(url) {  
  
  const [data, setData] = useState(null);  
  
  const [loading, setLoading] = useState(true);  
  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
  
    let cancelled = false;  
  
    fetch(url)  
  
      .then(r => r.json())  
  
      .then(d => { if (!cancelled) setData(d); })  
  
      .catch(e => { if (!cancelled) setError(e); })  
  
      .finally(() => { if (!cancelled) setLoading(false); });  
  
    return () => { cancelled = true; }; // cleanup!  
  
  }, [url]);  
  
  return { data, loading, error };  
  
}
```

■ FOLLOW-UP:

Why do we need the cancelled flag? What race condition does it prevent?

36

HIGH

[Performance] Optimize a list component using useMemo, useCallback, and React.memo.

CODE

```
// Parent:

const handleDelete = useCallback((id) => {
  setItems(prev => prev.filter(i => i.id !== id));
}, []); // stable reference – won't recreate on re-render

const sortedItems = useMemo(() =>
  [...items].sort((a, b) => a.name.localeCompare(b.name)),
  [items]); // recalculate only when items change

// Wrap child with React.memo:

const Item = React.memo(({ data, onDelete }) => (
  <div onClick={() => onDelete(data.id)}>{data.name}</div>
));
```

■ FOLLOW-UP:

What would happen without useCallback here? How many times would Item re-render?

37

HIGH

[Hooks] Implement a debounce search input using `useEffect` and `useState`.

CODE

```
function SearchBox() {  
  const [query, setQuery] = useState('');  
  const [debouncedQuery, setDebouncedQuery] = useState('');  
  
  useEffect(() => {  
    const timer = setTimeout(() => {  
      setDebouncedQuery(query);  
    }, 500);  
  
    return () => clearTimeout(timer); // cleanup on each keystroke!  
  }, [query]);  
  
  useEffect(() => {  
    if (debouncedQuery) fetchResults(debouncedQuery);  
  }, [debouncedQuery]);  
  
  return (  
    <input value={query} onChange={e => setQuery(e.target.value)} />  
  );  
}
```

■ FOLLOW-UP:

How would you convert this into a reusable `useDebounce` hook?

■ TRICKY [Bug Fix] BUG FIX: Why does this counter only increment by 1 instead of 2?

CODE

```
// ■ BUG – both calls read the same stale 'count' value:
```

```
const handleClick = () => {
```

```
  setCount(count + 1); // count = 0, sets to 1
```

```
  setCount(count + 1); // count = 0, sets to 1 again!
```

```
};
```

```
// ■ FIX – use functional update form:
```

```
const handleClick = () => {
```

```
  setCount(prev => prev + 1); // reads latest: 0→1
```

```
  setCount(prev => prev + 1); // reads latest: 1→2
```

```
};
```

```
// Now clicking increments by 2 correctly.
```

■ FOLLOW-UP:

When must you ALWAYS use the functional form of setState?

CODE

```
function useLocalStorage(key, initialValue) {  
  const [value, setValue] = useState(() => {  
    try {  
      const item = window.localStorage.getItem(key);  
      return item ? JSON.parse(item) : initialValue;  
    } catch { return initialValue; }  
  });  
  
  const setStoredValue = (newValue) => {  
    setValue(newValue);  
    window.localStorage.setItem(key, JSON.stringify(newValue));  
  };  
  
  return [value, setStoredValue];  
}  
  
// Usage:  
const [theme, setTheme] = useLocalStorage('theme', 'dark');
```

■ FOLLOW-UP:

How would you sync this across browser tabs using the storage event?

CODE

```
// ■ BUG – userId used inside but NOT in dependency array:

useEffect(() => {

  fetchUser(userId).then(setUser);

}, []); // stale closure! userId changes won't trigger re-fetch
```

```
// ■ FIX – add userId to deps + handle race condition:
```

```
useEffect(() => {

  let cancelled = false;

  fetchUser(userId).then(u => {

    if (!cancelled) setUser(u);

  });

  return () => { cancelled = true; };

}, [userId]); // re-runs whenever userId changes
```

■ FOLLOW-UP:

What ESLint rule catches this automatically? What plugin provides it?

41

HIGH

[Performance] **SCENARIO:** Your React app renders 10,000 list items and it's lagging. What do you do?

CODE

```
// Strategy 1: Virtual scrolling

// Use react-window – only renders visible rows in viewport

import { FixedSizeList } from 'react-window';

<FixedSizeList height={600} itemCount={10000} itemSize={35}>
  ({ index, style }) => <Row index={index} style={style} />
</FixedSizeList>

// Strategy 2: React.memo on list items

const Row = React.memo(({ data }) => <div>{data.name}</div>);

// Strategy 3: useMemo for filtered/sorted data

const filtered = useMemo(() => items.filter(i => i.active), [items]);

// Strategy 4: React 18 useTransition for non-urgent updates

const [isPending, startTransition] = useTransition();

startTransition(() => setQuery(newQuery)); // keeps typing responsive
```

■ FOLLOW-UP:

How does react-window work under the hood? What is 'windowing'?

■ QUICK REVISION

Key React concepts in 1–2 lines — perfect for last-minute review.

■ Virtual DOM

Lightweight JS copy of real DOM. React diffs it (reconciliation) and updates only changed parts.

■ Hooks Rules

Only call hooks at the top level. Never inside loops, conditions, or nested functions.

■ useMemo vs useCallback

useMemo → memoizes value. useCallback → memoizes function. Both take dependency arrays.

■ useEffect Deps

[] = run once. [dep] = run on dep change. No array = run every render. Return fn = cleanup.

■ Stale Closure

useEffect captures old values if deps are missing. ESLint's react-hooks/exhaustive-deps catches this.

■ React 18 Batching

All setState calls are auto-batched in React 18, even in setTimeout and promises.

■ React.memo

Wraps a component to skip re-renders if props haven't changed (shallow comparison).

■ Fiber

React's reconciler (v16+). Breaks work into units, enabling pausing, prioritization, Concurrent Mode.

■ Keys in Lists

Keys help React identify changed elements. Never use index as key if list can be reordered.

■ Context API

Global state without prop drilling. Every consumer re-renders on value change — use sparingly.